

POLITECHNIKA ŁÓDZKA

WYDZIAŁ FIZYKI TECHNICZNEJ, INFORMATYKI
I MATEMATYKI STOSOWANEJ

NAJWIEKSZE SKOJARZENIA W GRAFACH DWUDZIELNYCH

Paulina Koza
Nr albumu: 263311

ŁÓDŹ, CZERWIEC 2026

1 Wstęp

Problem znajdowania największego skojarzenia w grafie dwudzielnym jest jednym z klasycznych zagadnień teorii grafów oraz optymalizacji dyskretniej. Polega on na takim dobraniu par w grafie dwudzielnym, aby maksymalizować liczbę połączeń pomiędzy wierzchołkami dwóch rozłącznych zbiorów, przy założeniu, że każdy wierzchołek może być użyty co najwyżej raz.

W niniejszym projekcie problem ten zostanie przedstawiony na praktycznym przykładzie dobierania par do poloneza na studniówce. Uczniowie dzielą się na dwie grupy: chłopców i dziewczęta, a celem jest utworzenie jak największej liczby zgodnych par.

2 Opis problemu

Graf dwudzielnym to graf, którego zbiór wierzchołków można podzielić na dwa rozłączne zbiory, w naszym przypadku będzie to zbiór dziewcząt i zbiór chłopców. Dodatkowo mamy sytuacje największych skojarzeń co oznacza, że z jednego wierzchołka może wychodzić tylko jedna krawędź, oraz tworzymy możliwie najwięcej par. To również pasuje do naszego przykładu poloneza na studniówce.

Rozważmy więc graf dwudzielnym $G = (D, C)$, gdzie:

- D - zbiór dziewcząt,
- C - zbiór chłopców.

Każda krawędź oznacza, że dana para osób zgadza się zatańczyć razem poloneza. Celem jest znalezienie takiego zbioru krawędzi, w którym żadna osoba nie występuje więcej niż raz, a liczba par jest maksymalna.

3 Opis algorytmu

Program został napisany w języku Python i wykorzystuje algorytm wyszukiwania największego skojarzenia w grafie dwudzielnym przy użyciu przeszukiwania w głąb (DFS – Depth First Search).

W przedstawionym rozwiązaniu graf reprezentowany jest za pomocą listy sąsiedztwa. Wierzchołki jednego zbioru odpowiadają dziewczętom, natomiast wierzchołki drugiego zbioru odpowiadają chłopcom. Krawędź pomiędzy dziewczęciem a chłopcem oznacza możliwość utworzenia pary do poloneza.

Algorytm kolejno analizuje preferencje poszczególnych dziewcząt i próbuje przypisać im partnerów. Jeżeli wybrany chłopiec jest już przypisany, program sprawdza możliwość przeorganizowania istniejących skojarzeń w taki sposób, aby zwiększyć całkowitą liczbę utworzonych par. Dzięki temu otrzymywane jest największe możliwe skojarzenie dla zadanego grafu.

Poniżej przedstawiono implementację programu wraz z komentarzami objaśniającymi działanie poszczególnych fragmentów kodu.

4 Implementacja (Python)

```
class BipartiteMatching:

    def __init__(self):          #dziewczyna -> lista wybranych chlopcow

        self.graf = {}

    def dodaj_wybor(self, dziewczyna, chlopiec):
        #dodaje informacje o tym, ktorego chlopca wybrala dana dziewczyna

        if dziewczyna not in self.graf:
            self.graf[dziewczyna] = []

        self.graf[dziewczyna].append(chlopiec)

    def dfs(self, dziewczyna, odwiedzone):
        #szuka mozliwosci przypisania chlopca do dziewczyny (czy jest wolny)

        for chlopiec in self.graf.get(dziewczyna, []):

            if chlopiec in odwiedzone:
                continue

            odwiedzone.add(chlopiec)

            if chlopiec not in self.para_chlopiec:

                self.para_chlopiec[chlopiec] = dziewczyna
                return True

            poprzednia_dziewczyna = self.para_chlopiec[chlopiec]

            if self.dfs(poprzednia_dziewczyna, odwiedzone):

                self.para_chlopiec[chlopiec] = dziewczyna
                return True

        return False

    def najwieksze_skojarzenie(self):          #uruchamia algorytm dla
        kazdej dziewczyny i stopniowo zwieksza liczbe znalezionych par

        self.para_chlopiec = {}

        licznik = 0
```

```

for dziewczyna in self.graf:

    odwiedzone = set()

    if self.dfs(dziewczyna, odwiedzone):
        licznik += 1

return licznik, self.para_chlopiec

```

5 Analiza działania programu

Przedstawiona implementacja algorytmu największego skojarzenia w grafie dwudzielnym działa w oparciu o przeszukiwanie w głąb (DFS) oraz technikę ścieżek powiększających. Program iteracyjnie próbuje przypisać każdej dziewczynie odpowiedniego chłopca zgodnie z listą preferencji.

Jeżeli dany chłopiec jest już przypisany do innej dziewczyny, algorytm sprawdza możliwość zmiany wcześniejszych dopasowań tak, aby zwiększyć całkowitą liczbę par. Dzięki temu możliwe jest uzyskanie rozwiązania optymalnego, czyli największego skojarzenia.

6 Zastosowanie w problemie studniówki

W kontekście poloneza: wierzchołki to uczniowie, krawędzie to akceptowane pary, skojarzenie to przypisanie partnerów do tańca.

Celem jest: maksymalizacja liczby zadowolonych par, uniknięcie konfliktów (jedna osoba tylko w jednej parze).

7 Przykłady użycia kodu

```

#przyklad 1 - pelne skojarzenie

bm = BipartiteMatching()

bm.dodaj_wybor("D1", "C1")
bm.dodaj_wybor("D2", "C2")
bm.dodaj_wybor("D3", "C3")

liczba_par, wynik = bm.najwieksze_skojarzenie()

print("Liczba par:", liczba_par)

for chlopiec, dziewczyna in wynik.items():
    print(dziewczyna, "-", chlopiec)

```

Wynik po uruchomieniu:

Liczba par: 3
D1 - C1
D2 - C2
D3 - C3

```
#przyklad 2 - konflikt
```

```
bm = BipartiteMatching()
```

```
bm.dodaj_wybor("D1", "C1")  
bm.dodaj_wybor("D2", "C1")
```

```
liczba_par, wynik = bm.najwieksze_skojarzenie()
```

```
print("Liczba par:", liczba_par)
```

```
for chlopiec, dziewczyna in wynik.items():  
    print(dziewczyna, "-", chlopiec)
```

Wynik po uruchomieniu:

Liczba par: 1
D1 - C1

```
#przyklad 3 - wiecej dziewczat
```

```
bm = BipartiteMatching()
```

```
bm.dodaj_wybor("D1", "C1")  
bm.dodaj_wybor("D2", "C2")  
bm.dodaj_wybor("D3", "C1")  
bm.dodaj_wybor("D4", "C2")
```

```
liczba_par, wynik = bm.najwieksze_skojarzenie()
```

```
print("Liczba par:", liczba_par)
```

```
for chlopiec, dziewczyna in wynik.items():  
    print(dziewczyna, "-", chlopiec)
```

Wynik po uruchomieniu:

Liczba par: 2
D1 - C1
D2 - C2

```
#przyklad 4 - skojarzenia

bm = BipartiteMatching()

bm.dodaj_wybor("D1", "C1")
bm.dodaj_wybor("D1", "C2")

bm.dodaj_wybor("D2", "C1")

liczba_par, wynik = bm.najwieksze_skojarzenie()

print("Liczba par:", liczba_par)

for chlopiec, dziewczyna in wynik.items():
    print(dziewczyna, "-", chlopiec)
```

Wynik po uruchomieniu:

```
Liczba par: 2
D2 - C1
D1 - C2
```

```
#przyklad 5

bm = BipartiteMatching()

bm.dodaj_wybor("D1", "C1")
bm.dodaj_wybor("D1", "C2")

bm.dodaj_wybor("D2", "C1")

bm.dodaj_wybor("D3", "C2")
bm.dodaj_wybor("D3", "C3")

bm.dodaj_wybor("D4", "C3")
bm.dodaj_wybor("D4", "C4")

liczba_par, wynik = bm.najwieksze_skojarzenie()

print("Liczba par:", liczba_par)

for chlopiec, dziewczyna in wynik.items():
    print(dziewczyna, "-", chlopiec)
```

Wynik po uruchomieniu:

```
Liczba par: 4
D2 - C1
D1 - C2
```

D3 - C3

D4 - C4

```
#przyklad 6 - bez krawedzi

bm = BipartiteMatching()

#nie dodajemy krawedzi

liczba_par, wynik = bm.najwieksze_skojarzenie()

print("Liczba par:", liczba_par)

for chlopiec, dziewczyna in wynik.items():
    print(dziewczyna, "-", chlopiec)
```

Wynik po uruchomieniu:

Liczba par: 0

8 Wnioski

Celem niniejszego projektu było przedstawienie problemu doboru par do poloneza na studiówce jako zadania optymalizacji dyskretnej sprowadzonego do problemu największego skojarzenia w grafie dwudzielnym.

W pracy zaprezentowano model matematyczny opisujący relacje pomiędzy dziewczętami i chłopcami, gdzie krawędzie grafu odpowiadają akceptowalnym parom tanecznym, oraz implementację algorytmu opartego na przeszukiwaniu w głąb (DFS) i metodzie ścieżek powiększających.

Przeprowadzone testy i przykłady potwierdzają poprawność działania algorytmu zarówno w prostych przypadkach, jak i w sytuacjach wymagających reorganizacji już istniejących skojarzeń. Wyniki pokazują, że metoda skutecznie maksymalizuje liczbę możliwych par oraz znajduje rozwiązanie optymalne dla grafu dwudzielnego.

Najważniejszym elementem algorytmu jest mechanizm ścieżek powiększających, który umożliwia zwiększanie liczby skojarzeń poprzez modyfikację wcześniej utworzonych dopasowań.

Podsumowując, zastosowany algorytm DFS z metodą ścieżek powiększających stanowi skuteczne i intuicyjne narzędzie do rozwiązywania problemu największego skojarzenia, a jego implementacja dobrze ilustruje praktyczne zastosowanie teorii grafów w problemach organizacyjnych.